AGENTIC AI FOUNDATIONS ASSOCIATE

1Z0-1157-26 Certification Practice Exam

Exam Prep Guide: This practice companion features 12 exam-realistic multiple-choice questions mapped directly to the core OCI Agentic AI syllabus. These questions test your understanding of foundational architectural components (Brain, Hands, and Nervous System), reasoning loops (CoT, ReAct, and ToT), programmatic LangChain implementation, security threat modeling, and defense-in-depth guardrails. Use this document to assess your readiness for the 1Z0-1157-26 exam. *The answer key with detailed explanations is provided starting on Page 3.*

PART I: PRACTICE QUESTIONS

Question 1: What is the core execution loop difference between a standalone LLM and an LLM-based AI Agent?

A) Standalone LLMs are stateful, whereas AI Agents are entirely stateless and cannot persist conversational context.

B) Standalone LLMs operate on a single-pass inference response, whereas AI Agents execute iteratively inside a cyclical loop (Perceive → Reason → Act → Observe).

C) Standalone LLMs require real-time tools for basic text generation, whereas AI Agents are restricted to pre-trained static knowledge.

D) AI Agents run on local client-side servers, whereas standalone LLMs are strictly cloud-hosted.

Question 2: In the 'Two-tier LLM architecture pattern' for AI Agents, what are the respective roles of the models?

A) The primary model handles hard-coded safety guardrails, while the secondary model manages user registration.

B) A cheap, low-latency model is used for routine routing and tool dispatching, while a high-capacity model is reserved for advanced reasoning tasks.

C) An open-source model performs vector embedding generation, while a proprietary model indexes the database.

D) Both tiers run concurrently to average out token cost and speed up API response times.

Question 3: During an agent's tool execution cycle, why is it considered a critical security practice to keep tool code execution outside of the LLM?

A) The LLM lacks the memory capacity to cache tool results.

B) Running code inside the LLM increases token costs exponentially.

C) The LLM never executes tool code directly; it only outputs structured text (e.g., tool-call JSON) which is validated and run by the backend application environment.

D) Standardizing execution inside the LLM violates the Model Context Protocol (MCP) spec.

Question 4: How does an agentic framework (such as LangChain or SmolAgents) communicate a registered function's capabilities to the LLM?

A) It uploads the compiled binary directly to the LLM's system prompt.

B) It converts the Python function's docstring and type hints into a structured JSON schema that the LLM reads.

C) It translates the Python code into SQL queries for execution.

D) It queries an external API catalog whenever the user submits a prompt.

Question 5: Which of the following describes a key limitation of standalone Chain-of-Thought (CoT) prompting in agentic tasks?

A) It is incapable of generating structured markdown or JSON outputs.

B) It relies entirely on internal knowledge, cannot run tools to look up real-time facts, and cannot self-correct against external reality.

C) It requires model fine-tuning and cannot be triggered via simple system prompts.

D) It can only handle mathematical equations and is useless for conversational queries.

Question 6: Why does the ReAct (Reasoning + Acting) framework significantly reduce hallucinations compared to standard prompting?

A) It fine-tunes the model on specific domain-focused databases.

B) It completely bypasses intermediate reasoning steps to output answers immediately.

C) It interleaves reasoning traces with tool Actions and grounds subsequent reasoning steps on real-world Observations.

D) It forces the model to run in a localized, offline environment with zero external access.

Question 7: In the orchestration layer (the agent's nervous system), which module is specifically responsible for managing short-term scratchpads and long-term persistent history?

A) Safety & Guardrails

B) State Machine

C) Memory Management

D) Tool Routing

Question 8: An attacker plants malicious invisible instructions in a public web document. When an agent retrieves this document during a RAG lookup, the instructions hijack the session to steal sensitive data. What type of vulnerability is this?

A) Runaway Execution

B) Direct Prompt Injection

C) Memory Poisoning

D) Indirect Prompt Injection

Question 9: An attacker poisons an agent's long-term storage, altering how the agent behaves in all future sessions. Which security risk does this exploit represent?

A) Indirect Prompt Injection

B) Memory Poisoning

C) Runaway Execution

D) Direct API Exfiltration

Question 10: Under the defense-in-depth framework, which layer of the guardrails architecture is responsible for intercepting and screening toxic, unauthorized, or private data (PII) before it is returned to the user?

A) Input Validation

B) Output Filtering

C) Tool Boundaries

D) Observability

Question 11: A development team is building a system where multiple specialist agents (e.g., a researcher, an auditor, and a copywriter) must collaborate as a cohesive unit. Which framework is specifically designed for multi-agent role-based workflows?

A) OpenAI Agents SDK

B) Hugging Face SmolAgents

C) CrewAI

D) Standalone Chain-of-Thought

Question 12: What is the primary purpose of the Model Context Protocol (MCP) in modern agentic architectures?

A) To bypass the application backend and let the LLM execute system shell commands directly.

B) To provide a standardized protocol for securely connecting LLMs with the data, prompts, and tools they need.

C) To fine-tune the weights of low-capacity routing models.

D) To compress JSON schemas to reduce total input token overhead.

PART II: ANSWER KEY & EXPLANATIONS

Question 1: Correct Answer: B

Explanation: A standalone LLM is a single-pass inference engine—it receives a prompt and outputs a response without any dynamic iteration. An AI Agent, however, operates inside a closed loop (Perceive → Reason → Act → Observe) that runs iteratively until a final goal is reached or a guardrail triggers a stop. The agent is capable of planning multi-step routes dynamically based on external data.

Question 2: Correct Answer: B

Explanation: The two-tier LLM architecture pattern is an optimization strategy to balance cost and latency. A small, fast, cheap model is utilized at the front-end to classify user intent, handle basic routing, and manage simple tool routing. When the agent detects that deep logical thinking or complex reasoning is required (e.g., multi-step planning or math), it routes the sub-task to a heavier, more expensive reasoning-focused model.

Question 3: Correct Answer: C

Explanation: Letting the LLM execute tool code directly is a massive security hazard and technically impossible for basic models. In safe architectures, the LLM is restricted to a text-in, text-out interface. It emits structured text (typically a JSON schema) expressing its *intent* to call a tool. The hosting application code intercepts this text, validates the input parameters, and executes the actual code inside a secure boundary safely separated from the model.

Question 4: Correct Answer: B

Explanation: Agentic frameworks leverage Python metadata to automate tool discovery. By parsing a function's docstring (which describes what the tool does) and its parameter type hints, the framework automatically compiles a JSON schema. This schema is appended to the system prompt of the LLM, enabling the model to match user queries to the specific tool's metadata.

Question 5: Correct Answer: B

Explanation: Standalone Chain-of-Thought (CoT) prompting enables step-by-step thinking, but it operates entirely within the model's static, internal parameter weights. If those weights contain incorrect information, the step-by-step reasoning will simply be a logical sequence leading to a confidently wrong answer. It lacks the mechanism to invoke external tools, fetch live data, or modify its plan based on external observations.

Question 6: Correct Answer: C

Explanation: The ReAct (Reasoning + Acting) pattern reduces hallucinations because it grounds its reasoning steps in real-world reality. Instead of relying solely on internal weights (like CoT), it performs an Action (calls a tool), receives an actual Observation (tool output), and then uses that Observation to form its next Thought. This interleaving cycle prevents the model from fabricating false links.

Question 7: Correct Answer: C

Explanation: Memory Management is the orchestration loop component that handles an agent's storage. It acts as a short-term scratchpad for the active execution run (tracking intermediate thoughts and observations) and connects to long-term data stores (e.g., semantic vectors, conversation history, databases) to maintain context across multi-turn sessions.

Question 8: Correct Answer: D

Explanation: This is a classic **Indirect Prompt Injection**. Unlike direct injection, where the user types malicious instructions directly into the chat bar, indirect injection occurs when the agent autonomously reads external data (such as a poisoned text file, email, or web page) containing invisible or disguised commands that hijack the LLM's active orchestration loop.

Question 9: Correct Answer: B

Explanation: Memory Poisoning occurs when an attacker writes malicious instructions into an agent's persistent long-term memory. Since the memory is retrieved across subsequent user sessions, the exploit persists silently and can affect future decisions, leading to ongoing vulnerabilities such as secret data exfiltration or state corruption.

Question 10: Correct Answer: B

Explanation: Under a layered defense-in-depth guardrail framework, **Output Filtering** is the final line of defense. It acts as a filter that screens all generated responses for toxic content, PII leaks, API keys, or compliance violations before the text is displayed to the user.

Question 11: Correct Answer: C

Explanation: CrewAI is specifically built around multi-agent team mechanics. It allows developers to define individual, role-based agents with custom tools and personas (e.g., Researcher, Auditor) and choreograph their execution as a collaborative 'crew' with structured hand-offs.

Question 12: Correct Answer: B

Explanation: The **Model Context Protocol (MCP)** is an open-standard protocol designed to simplify how LLMs connect with external data sources, prompts, and tools. Instead of custom, fragmented integration layers, MCP provides a secure, unified protocol for developers to feed rich context into LLMs.